Trabajo Práctico N° 3

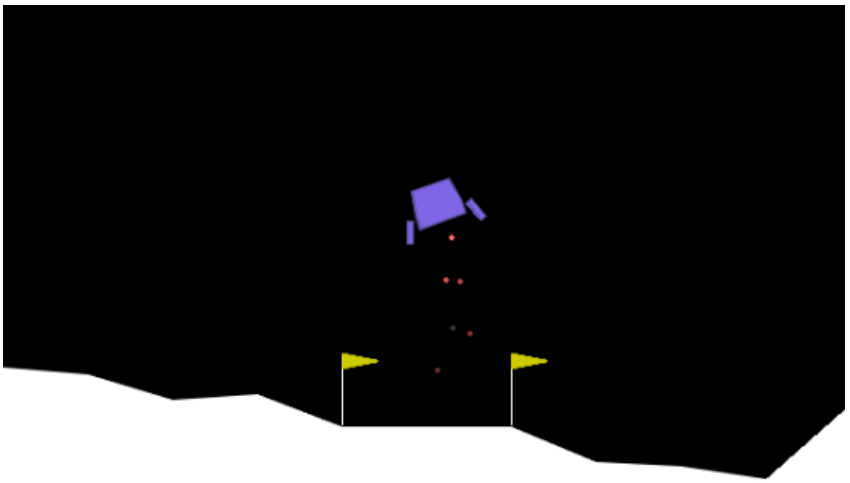
Aprendizaje por Refuerzo 1

CEIA — Cohorte 21Co2025 | Especialización en Inteligencia Artificial

INTEGRANTES
José Aviani - jose.aviani@gmail.com
José Luis Díaz - diazjoseluis@gmail.com
Ricardo Silvera - rsilvera@thalu.com.ar

---

1. El Entorno: LunarLander-v3



Este entorno simula el aterrizaje de un módulo lunar. El agente controla cuatro propulsores para descender suavemente sobre una plataforma de aterrizaje ubicada en el centro del terreno. El entorno finaliza cuando el módulo aterriza, choca o sale de los límites del escenario.

Descripción del entorno

Componente	Detalle
Espacio de observación	Continuo — 8 variables (`float32`)
Espacio de acciones	Discreto — 4 acciones

Componente	Detalle
Recompensa positiva	Aterrizaje suave: +100 (sobre la plataforma: +200)
Recompensa negativa	Crash, combustible consumido, desvío lateral
Condición de fin	Aterrizaje exitoso, crash o fuera de límites
Objetivo oficial	Promedio ≥ 200 puntos en 100 episodios consecutivos

Variables del estado

Variable	Índice	Significado
x	0	Posición horizontal
y	1	Posición vertical
v_x	2	Velocidad horizontal
v_y	3	Velocidad vertical
Θ	4	Ángulo de inclinación
ω	5	Velocidad angular
Contacto L	6	Pata izquierda toca suelo (0 / 1)
Contacto R	7	Pata derecha toca suelo (0 / 1)

Acciones disponibles

Acción	Significado
0	No hacer nada
1	Encender propulsor izquierdo
2	Encender propulsor principal (↑)
3	Encender propulsor derecho

Política aleatoria (baseline)

Para establecer una línea base se corrieron 50 episodios con política aleatoria, obteniendo:

Métrica	Valor
Media	**−171.1**
Rango	[−526.2, +108.0]

Este valor sirve como referencia mínima: cualquier política entrenada debe superar −171.1 para ser significativa.

---

2. Algoritmos Implementados

SARSA (on-policy TD)

SARSA actualiza la Q-table usando la acción que **realmente tomará** el agente en el siguiente paso, seleccionada con la política ε-greedy actual:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \, Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

donde a_{t+1} se obtiene de la política ε-greedy. Al actualizar con la acción que realmente se elegirá, SARSA es consciente del costo de la exploración y desarrolla una política más **conservadora**.

Q-Learning (off-policy TD)

Q-Learning actualiza la Q-table usando la acción **óptima hipotética**, independientemente de lo que el agente hará realmente:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Q-Learning aprende la política greedy óptima incluso mientras ejecuta una política exploratoria. Esto lo hace potencialmente más eficiente, pero también más inestable en entornos con penalizaciones severas.

---

3. Discretización del Espacio de Estados

SARSA y Q-Learning son algoritmos **tabulares**: requieren un espacio de estados finito y discreto para construir la tabla *Q[s, a]*. LunarLander tiene seis variables continuas, por lo que es necesario discretizarlas.

Método: se divide el rango de cada variable continua en n intervalos (*bins*) usando `numpy.digitize`, y se mapea cada combinación de intervalos a un índice entero único con `numpy.ravel_multi_index`.

Las dos variables de contacto (binarias) se incorporan directamente sin discretizar.

Evolución de la configuración de bins

A lo largo del desarrollo se probaron distintas configuraciones de discretización:

Configuración	Bins	Total de estados	Resultado
Uniforme 1	`N_BINS = 6` (igual para todas)	186.624	Granularidad gruesa — no distingue situaciones sutiles
Uniforme 2	`N_BINS = 10` (igual para todas)	4.000.000	Espacio demasiado grande — cobertura insuficiente con 80k episodios
No uniforme (v1)	`[6, 10, 8, 10, 8, 6]`	921.600	Cobertura de solo 1.34% — Q-table sin converger
No uniforme (v2)	`[4, 5, 3, 6, 4, 3]`	17.280	Buena cobertura con 200k episodios — configuración final
No uniforme (final)	**`[3, 5, 3, 6, 4, 3]`**	**12.960**	**↓ Más compacto → mayor cobertura y convergencia**

Configuración final

BINS_PER_VAR = [3, 5, 3, 6, 4, 3] # x, y, vx, vy, θ, ω

SHAPE = (3, 5, 3, 6, 4, 3, 2, 2)

N_STATES = 12.960

Q-table = 12.960 × 4 = 51.840 entradas (~0.4 MB en float64)

La asignación de bins es **no uniforme** y refleja la importancia de cada variable para la tarea de aterrizaje:

- `vy` (velocidad vertical) → 6 bins: variable más crítica para el aterrizaje suave
- `y` (altura) y `θ` (ángulo) → 5 y 4 bins respectivamente: relevantes para posicionamiento
- `x`, `vx`, `ω` → 3 bins: variables secundarias

Un espacio de estados más pequeño garantiza que cada estado sea visitado múltiples veces durante el entrenamiento, condición necesaria para la convergencia de los métodos tabulares.

---

4. Técnicas de Mejora

4.1 Inicialización optimista de la Q-table

En lugar de inicializar la tabla con ceros, se usa un valor positivo pequeño (`5.0`):

Q = np.full((N_STATES, N_ACTIONS), 5.0)

Con *Q = 0*, la primera acción explorada siempre parece "mala" respecto al estado inicial, lo que puede atrapar al agente en comportamientos subóptimos tempranos. La inicialización optimista incentiva al agente a probar todas las acciones antes de consolidar preferencias, mejorando la exploración inicial.

4.2 Reward Shaping potencial

Este escenario presenta un problema de **recompensa escasa**: la señal positiva significativa (+100/+200) solo aparece al aterrizar, pero el agente puede tardar cientos de pasos en llegar. Sin guía intermedia, el agente no asocia sus acciones con el éxito.

La solución es agregar una **recompensa densa artificial** derivada de un potencial de forma:

$$r_{\text{shaped}} = r + \gamma\,\Phi(s') - \Phi(s)$$

$$\Phi(s) = -\sqrt{x^2 + y^2} \;-\; 0.5\sqrt{v_x^2 + v_y^2} \;-\; 0.3\,|\theta|$$

Esta función penaliza distancia al pad, velocidad excesiva e inclinación. El teorema de Ng et al. (1999) garantiza que esta transformación **no modifica la política óptima** — solo acelera el aprendizaje al proporcionar gradiente en cada paso.

---

5. Hiperparámetros

Se utilizan los mismos valores para ambos algoritmos, permitiendo una comparación directa:

Parámetro	Valor	Descripción
`N_EPISODES`	200.000	Episodios de entrenamiento
`ALPHA` (α)	0.10	Tasa de aprendizaje — fracción de la diferencia TD que se incorpora
`GAMMA` (γ)	0.99	Factor de descuento — importancia de recompensas futuras
`EPSILON_START`	1.0	Exploración inicial 100% aleatoria
`EPSILON_END`	0.01	Mínimo de exploración del 1% al final
`EPSILON_DECAY`	0.9999	Decaimiento lento → exploración prolongada
`SEED`	42	Semilla de reproducibilidad

Criterio para los hiperparámetros:

- **ALPHA = 0.10**: un valor más bajo que en configuraciones anteriores (0.15–0.20) favorece actualizaciones suaves dado que con 12.960 estados cada uno se visita muchas veces.
- **EPSILON_DECAY = 0.9999**: decaimiento lento porque el espacio sigue siendo relativamente amplio y se necesita exploración sostenida. Con este valor, ε alcanza su mínimo recién alrededor del episodio 150.000.
- **GAMMA = 0.99**: alto, para que el agente valore el aterrizaje exitoso que ocurre decenas de pasos en el futuro.

---

6. Resultados del Entrenamiento

Evolución de la recompensa media (SARSA)

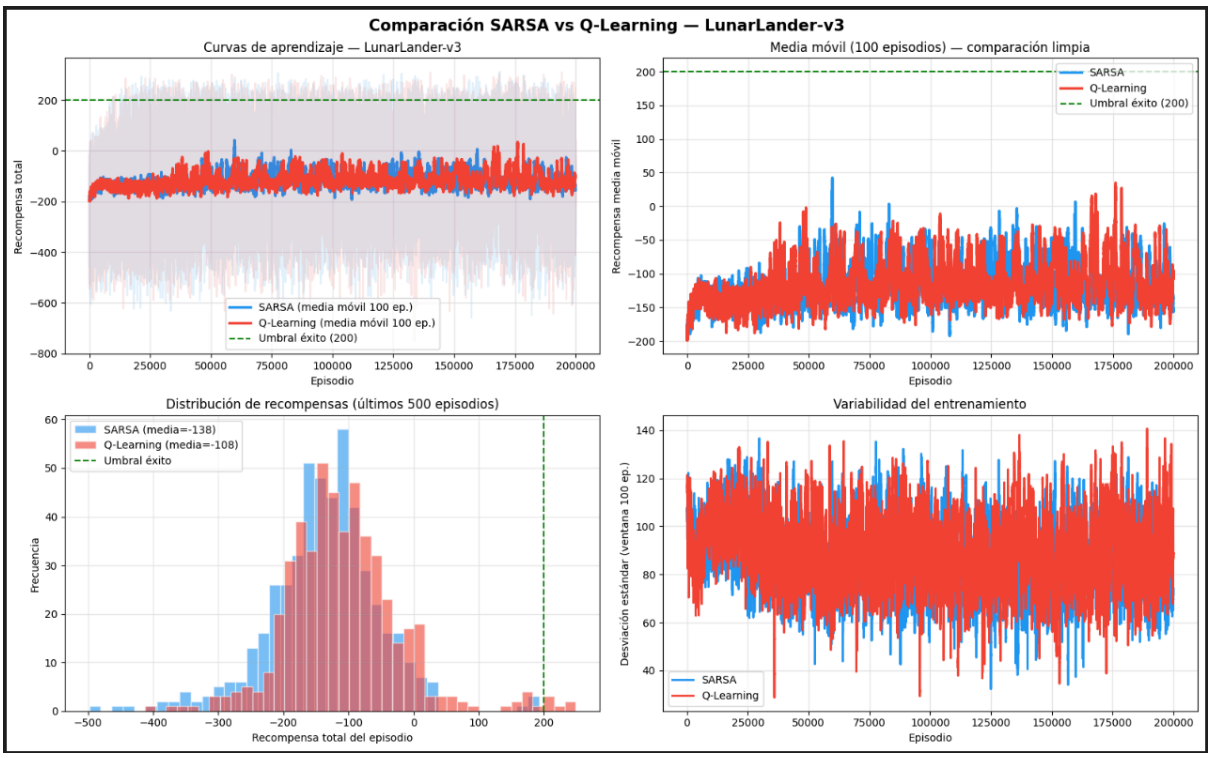
Episodio	Media últ. 100 ep.	ε	Estados visitados
500	−184.8	0.9512	2.26%
2.000	−156.8	0.8187	3.53%
5.000	−125.6	0.6065	4.81%
10.000	−133.4	0.3679	6.81%
200.000	**−154.9**	**0.0100**	**—**

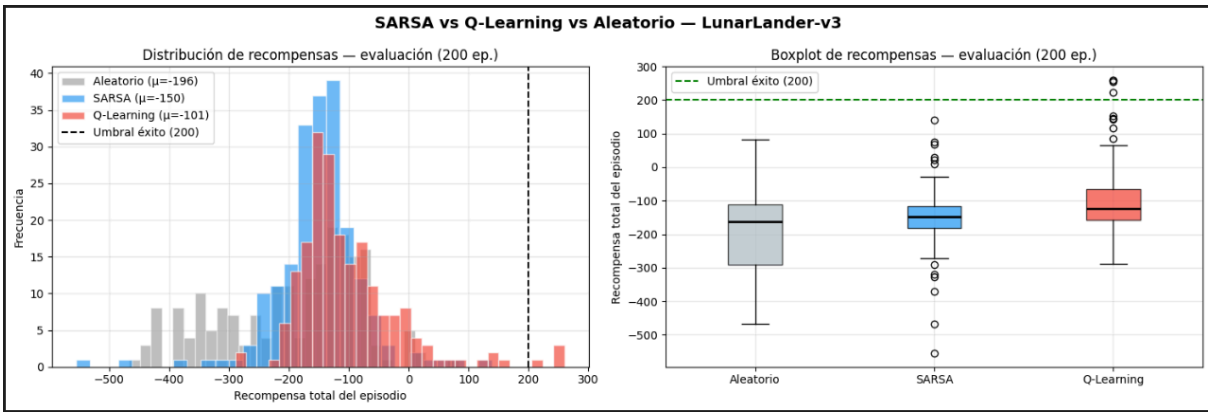
Evolución de la recompensa media (Q-Learning)

Episodio	Media últ. 100 ep.	ε	Estados visitados
500	−172.1	0.9512	2.15%
2.000	−146.6	0.8187	3.35%
5.000	−127.6	0.6065	4.81%
10.000	−126.9	0.3679	6.44%
200.000	**−98.4**	**0.0100**	**—**

Resumen final del entrenamiento

Algoritmo	Media últimos 100 ep.	Mejora vs. aleatoria
Política aleatoria (baseline)	−171.1	—
SARSA	−154.9	**+16.2 pts**
Q-Learning	**−98.4**	**+72.7 pts**





---

7. Evaluación de las Políticas Aprendidas

Las políticas entrenadas se evaluaron en **modo greedy puro** (ε = 0) sobre **200 episodios** con semillas distintas a las del entrenamiento, para medir generalización real.

Política	Media	Mediana	Máx.	Mín.	Tasa éxito (≥ 200)
Aleatoria	−196.2	−162.0	83.5	−467.1	0.0%
SARSA	−150.2	−147.2	139.5	−554.8	0.0%
Q-Learning	**−100.8**	**−124.5**	**260.2**	−289.3	**2.0%**

Observaciones:

- Q-Learning obtiene la mejor media (−100.8), alcanza puntajes por encima de 200 el 2% de los episodios (máximo: 260.2) y supera ampliamente a la política aleatoria.
- SARSA mejora sobre la aleatoria pero no logra ningún episodio exitoso en modo greedy puro.
- La política aleatoria obtiene incluso peor media en evaluación (−196.2) que durante el baseline de entrenamiento (−171.1)

---

8. Análisis y Conclusiones

Q-Learning superó a SARSA en esta configuración?

En configuraciones anteriores con exploración alta y espacio de estados mayor, SARSA tendía a ser más conservador y obtenía mejores resultados. Con el espacio reducido a 12.960 estados y 200.000 episodios, Q-Learning demuestra su ventaja off-policy: **aprende la función de valor óptima aunque ejecute una política exploratoria**, acumulando información sobre qué acciones son óptimas incluso cuando no las ejecuta.

Limitación estructural del enfoque tabular

Independientemente de la configuración de bins utilizada, el enfoque tabular enfrenta una limitación fundamental en LunarLander: **no generaliza entre estados similares**. Si el agente visita el estado gran parte de los estados, la Q-table no tiene ninguna información sobre gran parte en el espacio continuo.

Soluciones más efectivas

El siguiente paso natural es **Deep Q-Network (DQN)**, que reemplaza la tabla por una red neuronal que aproxima Q(s, a). Al interpolar entre estados continuos similares.